

Machine Learning to Identify Encryption Protocols

Rohan Tripathi, Vignesh Krishnan, Ritwika Dhal

September 2024

1 Abstract

We explored the intersection of cryptography and machine learning by asking: Can a computer identify which encryption method was used just by analyzing encrypted text? Using three widely-used encryption protocols—AES, DES, and RC5—we trained machine learning models, including XGBoost, Random Forest, and Linear SVC, to recognize patterns in ciphertext and examined how different types of input text affect classification accuracy.

Our findings were both intriguing and revealing. Simple, predictable text resulted in high accuracy, with Random Forest achieving up to 98%, while random characters dropped accuracy to around 62%. Interestingly, RC5 encryption was consistently easier to classify, while AES and DES often overlapped and were harder to distinguish. This project not only demonstrates the potential of machine learning in cryptography but also highlights vulnerabilities caused by predictable input patterns, emphasizing the critical role of plaintext characteristics in encryption analysis.

2 Introduction

When we began this project, we were captivated by a question: can machine learning uncover the secrets of encrypted data and identify which method was used? We chose three encryption protocols—AES, DES, and RC5—that represent milestones in the evolution of cryptography. Each has its own unique "personality": AES, the reigning champion of security; DES, the legacy workhorse still found in older systems; and RC5, the adaptable middle ground with its customizable parameters.

To evaluate these algorithms, we needed metrics that could do more than scratch the surface. Accuracy was our starting point—how often did our models get it right? But we didn't stop there. Precision and recall helped us dive deeper, showing where our models excelled or stumbled. The F1-score balanced these measures, and confusion matrices gave us a clearer view of where the models struggled, particularly between AES and DES. Some metrics didn't make the cut. For example, ROC-AUC curves weren't suited for our multiclass classification problem, and we chose not to measure computation time since our focus was on feasibility rather than speed.

What we built isn't just a tool to classify encryption methods—it's a lens into how these algorithms behave. Our work speaks to cryptographers and security analysts looking for patterns, researchers exploring this rich intersection of cryptography and machine learning, and students eager to learn about both fields. This isn't about breaking encryption; it's about understanding how encryption methods leave subtle fingerprints on data. Think of it like identifying artists by their brushstrokes—only here, our canvas is encrypted text, and the patterns tell a deeper story about the art of securing data.

Our encryption classification system offers practical applications across critical areas, from cybersecurity to education. In network security, it could act as an early warning system, flagging outdated encryption methods like DES and helping security teams identify vulnerabilities in network traffic. Forensic investigators might use it to analyze encryption methods during security incidents, while students and researchers could leverage it to visualize how encryption algorithms behave with various input types. For organizations

managing legacy systems, it simplifies identifying outdated encryption methods, making upgrade planning more efficient. The strength of our system lies in its ability to analyze encryption patterns without attempting to break the encryption—like identifying a lock’s make and model without picking it.

3 Description/Methodology

Our approach to identifying encryption protocols combined systematic data preparation with machine learning analysis, creating a pipeline that transforms plaintext into encryption predictions. Here’s how we built and tested our system.

Our approach to identifying encryption protocols combined systematic data preparation with machine learning analysis, creating a pipeline that transforms plaintext into encryption predictions. Here’s how we built and tested our system. We began by designing four distinct datasets to test our models under different conditions. Dataset D1 included simple, predictable vocabulary—basic sentences and common words. D2 introduced repeated string patterns, while D3 focused on shorter text snippets. D4, the toughest test case, contained completely random characters. Each plaintext entry was encrypted using AES, DES, and RC5, and we labeled every ciphertext with its corresponding encryption method. The encrypted data was then processed through a pipeline that converted hexadecimal ciphertext into numerical arrays, translating complex encrypted data into a format machine learning models could understand. Each ciphertext became a row in our feature matrix, paired with a label: AES=0, DES=1, RC5=2.

3.1 For classification, we selected three ML models:

- XGBoost
- Random Forest
- Linear SVC

We trained the models on an 80-20 split of training and testing data, allowing them to learn the patterns associated with each encryption method. To evaluate performance, we used metrics like accuracy, precision, recall, and F1-score. Confusion matrices were particularly insightful, revealing where models struggled to differentiate algorithms. For example, AES and DES were sometimes confused, highlighting overlaps in their encryption outputs.

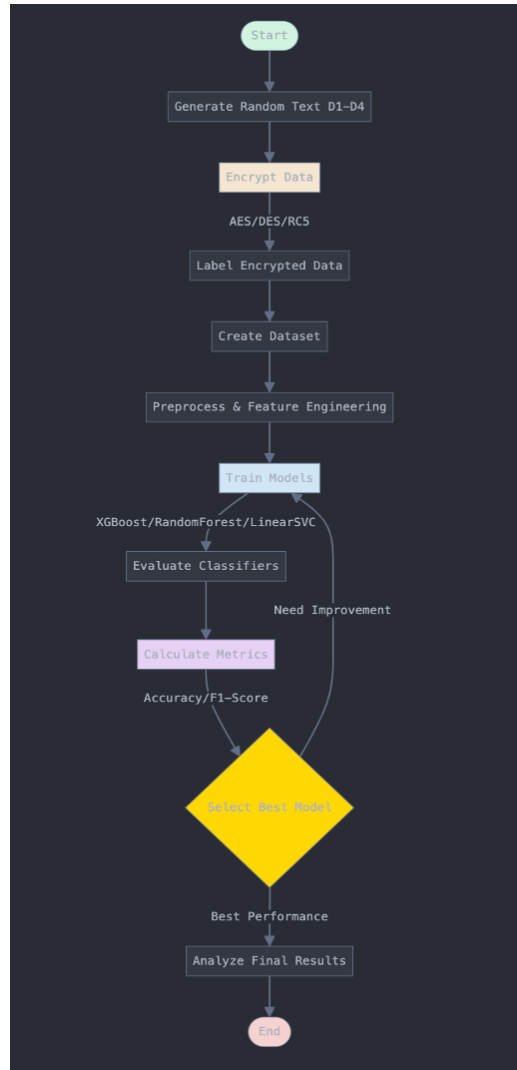
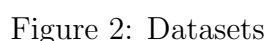


Figure 1: Flow of the Project

Here are snapshots of the plaintexts and random texts that users can provide as input to our encryption classification system. These inputs range from simple, predictable text to completely random strings, designed to test the system’s versatility.



5

D3 Cipher Texts		D1 Cipher Texts	
Ciphertext	Algorithm	Ciphertext	Algorithm
c591146ab51f6ca0b	AES	b9806ca40d8eab608fdcaae6a8t	AES
661a404f5eb1f4e96	DES	29674a2c702d911d02f3d65d97	DES
34e96bc36b41073a	RC5	34d62cedb9f19f4e7bf9dd06d17	RC5
c149748cabf773beb	AES	985e132888780dc59bbac45f18f	AES
faf7b3d1b0a364cad	DES	46ae4512772701df05dd6759414	DES
1afd7005dd0b27f2f6	RC5	03a59d16ed60f2f4f1de6963789f	RC5
ad149d9ed129b3a7	AES	1cc5ebcd09c3a09478ce216740	AES
c58b54b4dffbc8cc8	DES	d33ce25a124e4eb8ee621af3c4c	DES
		03a59d16ed60f2f4f1de6963789f	RC5
		92ed3bcd97c0b8e6f0a39d84dbc	AES
		cbd21fa2e92aed11a2d7128239f	DES

D2 Cipher Texts		D4 Cipher Texts	
Ciphertext	Algorithm	Ciphertext	Algorithm
c0768ca86b40284fd5t	AES	ff2604a488cbc67face4	AES
ca402d1bfe26e9ef894	DES	68ed74c87a71bcab29	DES
4d9c2fb150c73bc24d	RC5	829e4314e4bcd6380	RC5
2f30dc5be11390d426f	AES	f605b8dd447a8b62677	AES
b600ea2dfbbd2339ac	DES	ae8944c5663778a2c3	DES
2cc05cf43e0325762cc	RC5	925fd8a1e4c26509919	RC5
87a21cb19db5a4ec49	AES	080405407764074297	AES
c14c5a0fca418317ff67	DES	0a8ce86b840c0ac6de	DES

Figure 3: Cipher Texts

4 Results

Our investigation into machine learning-based encryption classification revealed fascinating insights into how encryption methods leave unique patterns in data. Performance varied widely across our datasets, with accuracy ranging from an impressive 98% to 62%, largely depending on the input text characteristics.

The Random Forest model emerged as the standout performer, achieving 98% accuracy with Dataset D3 (short strings). Its ability to handle complex patterns made it particularly effective in these scenarios. XGBoost also performed consistently well, delivering accuracy between 83-89% across most datasets. On the other hand, Dataset D4 (random characters) proved challenging for all models, with accuracy dropping to 62%, highlighting how plaintext randomness diminishes encryption method distinctiveness.

One of the most striking observations was the consistent ease of identifying RC5 encryption. Confusion matrices revealed minimal misclassification for

RC5, unlike AES and DES, which frequently got mixed up, particularly with more complex inputs. For example, Dataset D1 (simple vocabulary) produced 89% accuracy with XGBoost, while Dataset D2 (repeated strings) achieved 84% with Random Forest.

Accuracy: 0.89

Classification Report:				
	precision	recall	f1-score	support
AES	0.99	0.67	0.80	20162
DES	0.75	0.99	0.85	19857
RC5	1.00	1.00	1.00	19981
accuracy			0.89	60000
macro avg	0.91	0.89	0.88	60000
weighted avg	0.91	0.89	0.88	60000

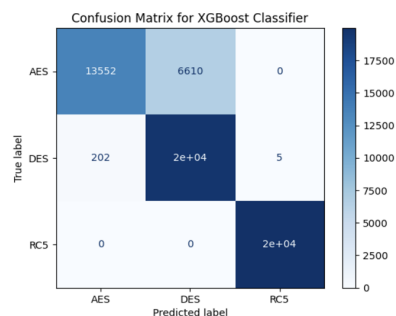


Figure 4: D1 Results

Accuracy: 0.84

Classification Report:				
	precision	recall	f1-score	support
AES	0.69	0.97	0.80	20162
DES	0.94	0.55	0.70	19857
RC5	1.00	1.00	1.00	19981
accuracy			0.84	60000
macro avg	0.88	0.84	0.83	60000
weighted avg	0.88	0.84	0.83	60000

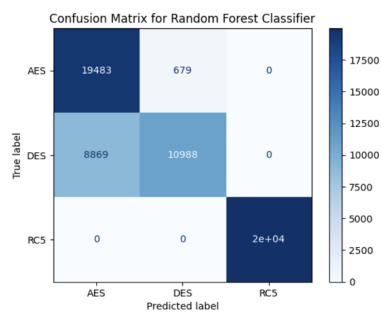


Figure 5: D2 Results

```

Cross-Validation Accuracy Scores: [0.83534 0.83206 0.84075]
Mean Accuracy: 0.84
Standard Deviation: 0.00

Test Set Evaluation:
Accuracy: 0.83
Classification Report:

```

	precision	recall	f1-score	support
AES	1.00	1.00	1.00	20162
DES	0.73	0.79	0.76	19857
RC5	0.77	0.71	0.74	19981
accuracy			0.83	60000
macro avg	0.84	0.83	0.83	60000
weighted avg	0.84	0.83	0.83	60000

```

Accuracy: 0.98
Classification Report:

```

	precision	recall	f1-score	support
AES	1.00	1.00	1.00	20162
DES	0.94	1.00	0.97	19857
RC5	1.00	0.94	0.97	19981
accuracy			0.98	60000
macro avg	0.98	0.98	0.98	60000
weighted avg	0.98	0.98	0.98	60000

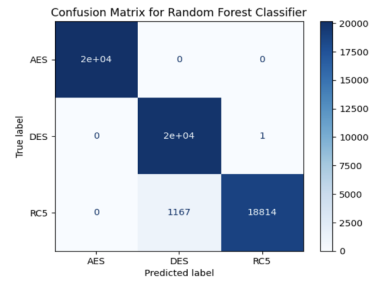
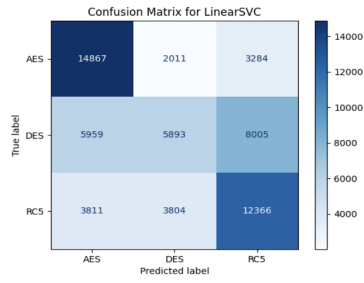


Figure 6: D3 Results

```

Accuracy: 0.62
Classification Report:

```

	precision	recall	f1-score	support
AES	0.60	1.00	0.75	20162
DES	0.57	0.42	0.49	19857
RC5	0.72	0.43	0.54	19981
accuracy			0.62	60000
macro avg	0.63	0.62	0.59	60000
weighted avg	0.63	0.62	0.59	60000

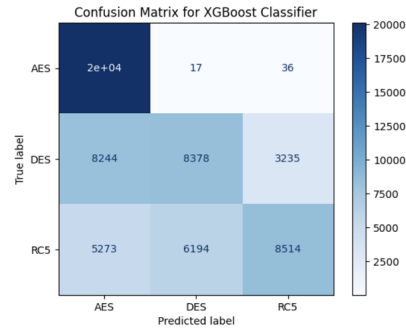


Figure 7: D4 Results

These results illuminate several challenges. Random character strings in Dataset D4 posed the biggest hurdle, emphasizing how encryption effectively obscures patterns when inputs are highly unpredictable. Additionally, padding in shorter inputs introduced unintended patterns that models

picked up, a finding we didn't anticipate. If we were to revisit this project, we'd focus on developing features to capture sequential patterns in ciphertext, as our current approach treats chunks independently. Exploring deep learning models like transformers could uncover subtle relationships that our current methods miss. These findings reinforce the importance of robust encryption practices. While high accuracy with predictable text might seem concerning, it highlights the importance of using random padding and avoiding predictable patterns in sensitive data. Encryption strength depends not only on the algorithm but also on how it's implemented—a critical takeaway for secure systems.

5 Contributions

- **Rohan Tripathi:** Handled dataset creation (D1–D4), encryption with AES, DES, and RC5, and preprocessing ciphertext into numerical arrays for machine learning.
- **Vignesh Krishnan:** Led model training and evaluation, optimizing Random Forest, XGBoost, and Linear SVC. Analyzed performance metrics and confusion matrices.
- **Ritwika Dhal:** Compiled results, created visualizations (diagrams and graphs), and prepared the presentation and report. Managed group discussions and ensured project coherence.

6 Conclusions

Lessons Learned: One of the biggest takeaways from this project was realizing how much the structure of input text affects the patterns in encrypted data. It showed us that machine learning can classify encryption methods effectively, but only when the input data has some consistency or predictability. We also learned how important it is to test models on a wide variety of data to really understand their strengths and weaknesses.

What Worked Well: Random Forest turned out to be a great choice—it handled the patterns in ciphertext really well, especially with simpler inputs. Designing the datasets (D1–D4) with different types of input text also worked out better than we expected. It gave us a clear picture of how different inputs impact encryption patterns. Using metrics like confusion matrices was helpful too; they gave us a way to pinpoint where and why the models were struggling.

Challenges: Random inputs (like in D4) gave us a tough time. The models couldn’t find clear patterns in completely random characters, which dropped accuracy a lot. Another tricky part was distinguishing between AES and DES; their outputs overlapped more than we thought. Something else we didn’t expect was how padding (extra bits added to fit block sizes) could affect results, especially with shorter inputs.

What We’d Do Differently: If we had to do this again, we’d probably spend more time on feature engineering—finding better ways to capture differences between similar encryption methods like AES and DES. Trying out more advanced models, like transformers, could also help with handling random inputs. Lastly, we’d experiment with even more diverse datasets to make the system more adaptable to real-world scenarios.

Appendix

Source Code:

The complete source code for this project is available at the following link:

https://colab.research.google.com/drive/1f_zunvgw-afZQEqsQUPkifaNFfiZ7TyZI